Thanks for providing that context! Based on Person A's frontend experience and desire to learn backend, and Person B's flexibility and comfort with Python, here's a recommended task split:

Recommended Option: A Variation of Splitting by Frontend/Backend with a Backend Focus for Person A

This approach allows Person A to focus on the backend development they want to gain experience in, while Person B can take on a mix of tasks, including some backend and potentially assisting with integrating frontend and backend components.

- **Person A: Core Backend & AI Logic Specialist**
    - **Primary Focus: Adapt all Python code from the Colab notebook (Sections 4, 5, 6, 7).** This is where the core logic for CV processing, embedding, vector database interaction, and AI prompting resides. This will give Person A significant backend experience.
    - **Set up the backend web framework.** Choose a Python framework like Flask, Django, or FastAPI and get the basic application structure running.
    - **Create the APIs** that the frontend will call (e.g., API endpoints for uploading a CV, submitting a job link, sending a message to the AI chat, etc.).
    - Manage the vector database setup and persistence in the backend environment.
    - Potentially lead the implementation of the job link research logic on the backend.
- **Person B: Integration, Frontend Support, and Supplementary Backend Tasks**
    - **Primary Focus: Build the web frontend.** Design and create the user interface for the application, including CV upload forms, the job link input, the AI chat interface, and displaying results.
    - **Connect the frontend to the backend APIs** developed by Person A. This involves making HTTP requests from the frontend to the backend.
    - **Assist with backend development as needed.** Person B can pair program with Person A on certain backend tasks, contribute to writing tests for the backend APIs, or take on specific backend features (like user authentication if you decide to add it later).
    - Refine the user experience and design.

Why this split works:

- **Aligns with Person A's Goal:** Person A gets hands-on experience with the core backend logic, AI integration, and API development.
- **Leverages Person A's Experience:** Person A's prior website development experience will be valuable in understanding the overall application structure and the interaction between frontend and backend.
- **Provides Flexibility for Person B:** Person B gets to work on the frontend, which is a crucial part of the user-facing application, and can also delve into backend tasks to broaden their skillset.
- **Encourages Collaboration:** The split naturally requires close collaboration, especially when defining and integrating the frontend and backend components.

Next Steps for Collaboration:

1. **Choose a Backend Framework:** As a pair, research Flask, Django, and FastAPI to decide which Python web framework you want to use. FastAPI is often a good choice for building APIs quickly.
2. **Set up Version Control:** Create a Git repository on GitHub/GitLab and ensure both of you are comfortable with basic Git commands (clone, branch, add, commit, push, pull, merge).
3. **Plan the API Endpoints:** Before Person A starts coding the backend logic, discuss and define the necessary API endpoints the frontend will need to interact with the backend. This provides a clear contract for both developers.
4. **Start with Core Functionality:** Begin by implementing the most critical parts adapted from the Colab notebook – perhaps starting with CV upload, processing, and indexing, then moving to retrieval and the basic chat interaction.
5. **Regular Check-ins:** Schedule regular (daily or every other day) brief meetings to discuss progress, challenges, and coordinate work.
6. **Pair Programming:** Consider pair programming sessions for complex parts of the logic or when integrating components.

This split should provide a good balance of responsibility while allowing Person A to focus on their backend learning goal and Person B to contribute significantly to the project's success.

Does this sound like a good approach for your team?